

ABB -

```
float* buscarABB(float* raiz, int valor) {
```

```
    if (raiz == NULL || raiz->info == valor) {
```

```
        return raiz;
```

```
    }
```

```
    if (valor < raiz->info) {
```

```
        return buscarABB(raiz->esq, valor);
```

```
    } else {
```

```
        return buscarABB(raiz->dir, valor);
```

```
    }
```

```
}
```

```
int contarNos(float* raiz) {
```

```
    if (raiz == NULL) {
```

```
        return 0;
```

```
    }
```

```
    return 1 + contarNos(raiz->esq) + contarNos(raiz->dir);
```

```
}
```

```
int altura(float* raiz) {
```

```
    int he, hd;
```

```
    if (raiz == NULL) {
```

```
        return -1;
```

```
    }
```

```
    he = altura(raiz->esq);
```

```
    hd = altura(raiz->dir);
```

```
    return 1 + (he > hd ? he : hd);
```

```
}
```

```
int contarFolhas (NoArv * raiz) {
```

```
    if (raiz == NULL) {
```

```
        return 0;
```

```
    }
```

```
    if (raiz->esq == NULL && raiz->dir == NULL) {
```

```
        return 1;
```

```
    }
```

```
    return contarFolhas (raiz->esq) + contarFolhas (raiz->dir);
```

```
}
```

```
int menorABD (NoArv * raiz) {
```

```
    while (raiz->esq != NULL) {
```

```
        raiz = raiz->esq;
```

```
    }
```

```
    return raiz->info;
```

```
}
```

```
void espelheArvore (NoArv * raiz) {
```

```
    NoArv * temp;
```

```
    if (raiz == NULL) {
```

```
        return;
```

```
    }
```

```
    temp = raiz->esq;
```

```
    raiz->esq = raiz->dir;
```

```
    raiz->dir = temp;
```

```
    espelheArvore (raiz->esq);
```

```
    espelheArvore (raiz->dir);
```

```
}
```

```
int countNodes(Node* root)
```

```
{ if (root == NULL)
```

```
    return 0;
```

```
}
```

```
return (countNodes(root->left) + countNodes(root->right) + countNodes(root->down));
```

```
}
```

```
int areIdentical(Node* a1, Node* a2)
```

```
{ if (a1 == NULL && a2 == NULL)
```

```
    return 1;
```

```
}
```

```
if (a1 == NULL || a2 == NULL)
```

```
    return 0;
```

```
}
```

```
return (a1->left == a2->left) && areIdentical(a1->right, a2->right) && (a1->down == a2->down);
```

```
}
```

```
int check(Node* root)
```

```
{ if (root == NULL)
```

```
    return 1;
```

```
}
```

```
if ((root->right == NULL && root->down != NULL) ||
```

```
(root->right != NULL && root->down == NULL))
```

```
    return 0;
```

```
}
```

```
return check(root->right) && check(root->down);
```

```
}
```